

Apuntes

Resumen ejecutivo

Introducción + párrafo de aporte

1 Marco teórico

[Que es la computacion serverless]

Computación sin servidor. Es la ejecución de aplicaciones sin gestionar servidores: una plataforma ejecuta, escala y factura el código según la demanda de cada momento. No implica que desaparezcan los servidores ni la operación, sino que su gestión pasa al proveedor (CNCF, 2018). Schleiersmith et al. (2021) la caracterizan por tres cualidades: abstracción de los servidores y de su operación, cobro por uso en vez de por reserva (sin cargo por recursos ociosos) y escalamiento automático desde cero. Tampoco es sinónimo de funciones: incluye los servicios de backend gestionados, o BaaS (Golec et al., 2024).

Modelos de ejecución. En las funciones como servicio (FaaS) la unidad es una función breve, activada por un evento o una solicitud HTTP, que el proveedor escala horizontalmente (CNCF, 2018; Leitner et al., 2019) y libera tras un periodo sin uso, lo que se llama escalamiento a cero (Golec et al., 2024). En los contenedores sin servidor la unidad es una imagen de contenedor y el equipo ya no aprovisiona ni escala máquinas virtuales (AWS Fargate), aunque no siempre hay escalamiento a cero: Cloud Run retira la última instancia cuando no hay solicitudes (Google Cloud Run), mientras que Fargate cobra hasta que la tarea termina (AWS Fargate, precios). El cómputo en el borde procesa en recursos ubicados entre la fuente de los datos y el centro de datos de la nube (Shi et al., 2016); este informe estudia su variante sin servidor (Aslanpour et al., 2021).

Modelo de referencia. Las instancias siempre encendidas son capacidad aprovisionada en forma continua (máquinas virtuales, o contenedores sobre nodos fijos) que se factura por tiempo encendido, se use o no (Ustiugov et al., 2021). Con cargas en ráfagas, esa capacidad queda subutilizada (Eismann et al., 2021).

[Como funciona el modelo serverless, porque es una opcion viable]

Dimensiones de comparación. El costo en serverless se calcula por consumo (invocaciones, tiempo de ejecución, memoria, CPU y transferencia de datos) (Ghorbian, 2026), de modo que el tiempo ocioso no se paga (Adzic, 2017); en el modelo siempre encendido es casi fijo para una capacidad dada. Esa diferencia define un volumen de equilibrio (secciones 2.3 y 4.2). La latencia es el tiempo entre el envío de una solicitud y su respuesta, suma de la espera en cola, la asignación de recursos y la ejecución (Golec et al., 2024). Al escalar a cero se agrega el arranque en frío, el retardo de preparar un entorno nuevo (Ustiugov et al., 2021), tratado en las secciones 2.2 y 4.1; el borde, en cambio, reduce la latencia de red al acercar el cómputo a la fuente de los datos (Golec et al., 2024). La complejidad operacional es el trabajo del equipo para desplegar, escalar, monitorear y mantener el sistema. El modelo sin servidor traslada parte de ese trabajo al proveedor, un motivo frecuente de adopción (Eismann et al., 2021), pero Leitner et al. (2019) documentan herramientas de

prueba y despliegue poco maduras y la necesidad de componer servicios del proveedor, a lo que se suman sus límites y la dependencia (secciones~2.4 y~2.6).

2.1 Modelos de ejecución y aislamiento

2.1 Modelos de ejecución y aislamiento

Además de la unidad que ejecuta (sección 2), cada modelo se distingue por cómo aísla a los clientes que comparten una máquina (Shafiei et al., 2022). Compartir servidores permite vender tramos cortos de capacidad que de otro modo quedaría ociosa (Liu & Niu, 2024), pero multiplica el costo de toda sobrecarga de aislamiento (Agache et al., 2020). Agache et al. (2020) agrupan los mecanismos en tres familias: los contenedores, que comparten el núcleo del anfitrión y obligan a transar entre seguridad y compatibilidad; la virtualización, que da a cada carga su propio núcleo y aísla con fuerza, pero ocupa más memoria y tarda segundos en arrancar (Wen et al., 2023); y el aislamiento por lenguaje, como los isolates de V8, que comparten un proceso a costa de compatibilidad y de exposición a ataques de canal lateral. En medio se ubican las microVM, máquinas virtuales mínimas, y gVisor, un núcleo en espacio de usuario que atiende las llamadas al sistema del contenedor (Wang et al., 2022).

En FaaS, AWS Lambda crea cada entorno en una microVM de Firecracker que no reutiliza entre funciones ni entre cuentas (AWS Lambda Security whitepaper); según sus creadores, Firecracker agrega menos de 5 MB por microVM, arranca en menos de 125 ms, ejecuta binarios Linux sin modificar y sostiene también a Fargate (Agache et al., 2020). Cloud Run, base de las Cloud Run functions (documentación de GCP), ofrece un entorno basado en gVisor, de arranque rápido pero que no emula todas las llamadas al sistema, y otro basado en microVM, totalmente compatible con Linux y con mejor CPU y red, pero de arranque más lento en algunos servicios (documentación de entornos de Cloud Run); ambos suman una capa de virtualización por hardware (documentación de seguridad de Cloud Run). En pruebas independientes, gVisor aisló mejor el rendimiento, pero penalizó más las llamadas al sistema y la red (Wang et al., 2022). En el borde, con nodos de pocos recursos donde VM y contenedores resultan pesados (Xie et al., 2021), Cloudflare Workers ejecuta miles de isolates por máquina, solo acepta JavaScript y WebAssembly (sección 2.5) y, contra Spectre, restringe relojes e hilos (documentación de seguridad de Cloudflare Workers). En el modelo siempre encendido, la frontera es la VM arrendada: compatibilidad total (Agache et al., 2020) y recursos reservados aunque estén ociosos (Liu & Niu, 2024).

ME FALTA EL PARRAFO 3 CALMAO

Ningún mecanismo gana en todos los frentes a la vez (Seguridad, compatibilidad, densidad y velocidad de arranque se transan entre sí), tal como concluyen tanto Agache et al (2020) y Wang et al (2022), que dicen textualmente que “ningun runtime es ventajoso en todos los aspectos”. Google mismo reconoce que el entorno de segunda generación (microVM) “generalmente rinde más bajo carga sostenida”, pero arranca más lento que el de gVisor en ciertos servicios, lo que da un criterio directo ligado a la condición de carga. Los Isolates de

v8 solo admiten JavaScript y WebAssembly, sin binarios nativos, mientras que Firecracker sí ejecuta binarios Linux sin modificar, lo que marca la restricción de compatibilidad de código. Wang et al (2022) también muestran que gVisor penaliza especialmente las llamadas al sistema, la red y los archivos pequeños. Y aunque Firecracker atanca en menos de 125 ms, Lambda igual mantiene un pool de microVM pre arrancadas porque ese tiempo no alcanza para el escalamiento en ráfaga. La elección entre contenedores MicroVM, isolates o VM completa depende del tipo de código que se corra (lenguaje/binarios/syscalls), del patrón de carga (ráfagas cortas versus sostenidas) y de si el costo de mantener recursos siempre listos compensa evitar la latencia de arranque,

2.2 Arranque en frío

El escalado a cero libera los entornos inactivos; según mediciones de 2021, AWS desaloja la mitad de sus contenedores cada 380 s [copik2021sebs]. Sin instancias disponibles, la plataforma debe reservar recursos, crear el entorno aislado (ver 2.1), cargar runtime y dependencias y ejecutar la inicialización [golec2025coldstart]: esa latencia es el arranque en frío; reutilizar una instancia preparada es un arranque en caliente. Crece con dependencias grandes y runtimes pesados como la JVM o .NET, frente a Python o Node.js [golec2025coldstart], y aparece también al escalar de N a N+1 instancias en ráfagas [giannini2022cpuboot].

Medirlo exige separar inicialización y ejecución comparando invocaciones frías y calientes, como hace *Invoc-Overhead* [copik2021sebs]. AWS Lambda registra esa *Init Duration* y, desde agosto de 2025, la factura en todas las configuraciones: el arranque en frío también es costo [gupta2025initbilling].

Las mitigaciones actúan sobre distintas partes del proceso y difieren en costo. La concurrencia aprovisionada lo evita con instancias ya inicializadas (AWS, Cloud Run, Azure Functions) [aws2023provisionedconcurrency, googlecloudrunmininstances, azurefunctionsflex], pero paga capacidad ociosa, acercándose al modelo siempre encendido (ver 2.3 y 3.2), y no cubre el paso de N a N+1, que Google aborda con *startup CPU boost* [giannini2022cpuboot]. Las instantáneas, como SnapStart, se saltan la inicialización restaurando una captura cifrada de la microVM [aws2022snapstart], aunque parte del retraso pasa a la ejecución al cargarse la memoria desde disco [ustiugov2021snapshots]. Los entornos livianos (V8, WebAssembly; ver 2.5) abaratan la creación del entorno: en un prototipo, Wasm redujo el arranque en frío cerca de un 94 % frente a Docker [gackstatter2022pushing], pero exige adaptar el código.

El impacto depende del patrón de carga [copik2021sebs]. En APIs síncronas, donde degrada la experiencia de usuario, sirven las instantáneas (con costo de caché en Python y .NET) [aws2022snapstart] o los aislamientos livianos, cambiando de plataforma [gackstatter2022pushing]; aun así, Cloudflare afirmó en 2020 haberlo eliminado [cloudflare2020workers], pero reapareció con scripts más pesados y su técnica de sharding lo redujo 10 veces [cloudflare2025shard]. En tráfico sostenido, la concurrencia aprovisionada es más económica sobre el 60 % de uso continuo [gupta2025initbilling]. En procesos asíncronos, fuera de la ruta crítica, la latencia es tolerable: es despreciable en

funciones de más de 10 s y, sin alta utilización, escalar a cero cuesta menos que una máquina virtual [copik2021sebs].

Las mediciones propias del grupo (sección 4.1) [COMPLETAR: confirman / matizan] este orden de magnitud: [COMPLETAR: frío vs. caliente por plataforma FALTA SEGUN LO QUE HAGA EL ARRATIA].

2.3 Facturación por consumo

Según \cite{lin_demystifying_2026}, la factura de una plataforma sin servidor combina cuatro factores: el tiempo de reloj facturable, los recursos asignados o consumidos durante ese tiempo, la granularidad o el mínimo de cobro y un cargo fijo por invocación. Las funciones cobran invocaciones y duración ponderada por la memoria asignada; AWS Lambda redondea al milisegundo y, desde agosto de 2025, factura también la fase de inicialización

\cite{Shubham_aws_2025}, mientras que Azure Functions ofrece un plan Premium con instancias precalentadas cobrado por núcleo-segundo y memoria

\cite{microsoft2026functionsscale}. Los contenedores sin servidor cobran vCPU y memoria por segundo: Fargate con un mínimo de un minuto por tarea

\cite{aws_precios2026} y Cloud Run por solicitud o por instancia, a elección

\cite{google2026cloudrunpricing}. En el borde, Cloudflare Workers cobra solicitudes y tiempo de CPU consumido, no de reloj, y no cobra transferencia de datos \cite{aws_cloudflare_2026}. Los valores numéricos se remiten al Cuadro comparativo de plataformas serverless y de borde de la sección 3.2.

Como cada modelo mide una unidad distinta, la CPU facturada es entre 1,01 veces (Workers) y 3,63 veces (Google) la consumida, según trazas de Huawei Cloud \parencite{lin_demystifying_2026}. El cobro por reloj incluye la espera por E/S (por ejemplo, esperar a que responda una base de datos), y encadenar funciones duplica esa espera \parencite{eismann2021whywhenhow}. En Lambda, Google e IBM incluyen también la inicialización \parencite{lin_demystifying_2026}. Workers cobra la CPU consumida sin contar la duración, con 128 megabytes fijos \parencite{cloudflare2026workerslimits}. Por eso las unidades solo se comparan normalizadas.

El punto de equilibrio es la carga desde la cual pagar por consumo cuesta más que una instancia siempre encendida. Pagar por consumo conviene mientras el costo de precio sea menor que la razón entre tiempo contratado y tiempo usado

\parencites{xiayue_charles_lin_serverless_2020} \parencites{copik2021sebs}. El cruce llega antes con carga regular que en ráfagas, y todos los patrones cruzan el punto de equilibrio si el volumen crece \parencite{liu2024demystifying}. La capacidad fija solo conviene si se mantiene una alta utilización \parencite{copik2021sebs}. Sin variar el volumen, el cruce se desplaza con los descuentos por compromiso (hasta 50\% en Fargate y 17\% en Lambda, revisar la tabla~\ref{tab:criterios}), la concurrencia aprovisionada

\parencite{lin_demystifying_2026} y costos como mantenimiento y egreso

\parencite{hamza2024costdynamics}. La evidencia es frágil: dos casos anecdóticos

\parencite{gojko_adzic_serverless_2017}, precios de 2020

\parencite{xiayue_charles_lin_serverless_2020} \parencite{copik2021sebs} y trazas de un

solo proveedor \parencite{lin_demystifying_2026}. El detalle de su cuantificación está en la sección~\ref{sec:costos}

2.4 Diseño orientado a eventos

ayayayayaayay

Presupuesto: 0,75 páginas · 300–350 palabras · tres párrafos

Párrafo 1 — Concepto *(qué es el diseño orientado a eventos, en términos que sirvan para comparar después)*

[COMPLETAR: definición de arquitectura orientada a eventos. Cubrir: colas vs. pub/sub, idempotencia, semántica de entrega al menos una vez (at-least-once), orquestación vs. coreografía. Cita: [INVESTIGAR: fuente académica/CNCF].]

Párrafo 2 — Comparación entre modelos *(cómo cambia el diseño orientado a eventos según el modelo de ejecución: FaaS, contenedores sin servidor, borde, instancias always-on)*

[COMPLETAR: en FaaS el evento dispara la función directamente (integración nativa con colas/pub-sub del proveedor); en contenedores sin servidor normalmente hace falta un runtime propio escuchando la cola; en el borde el modelo de eventos es distinto (más cercano a request/response que a colas); en always-on el consumidor vive permanentemente escuchando. [INVESTIGAR: confirmar con documentación oficial de al menos un producto por modelo].]

Párrafo 3 — Cuándo conviene *(cierre obligatorio de la sección, en términos de costo / latencia / complejidad operacional)*

[COMPLETAR: bajo qué condiciones de carga y de tipo de evento conviene cada modelo para arquitecturas orientadas a eventos. Debe responder explícitamente la pregunta comparativa de la ficha.]

2.5 WebAssembly y WASI

WebAssembly (Wasm) es una abstracción portable sobre el hardware, de ejecución eficiente, segura e independiente de la plataforma [1] [2]. Aísla cada módulo en una memoria lineal separada del código y de la pila, y en arquitecturas de 64 bits usa páginas de guarda que delegan la verificación de límites al hardware (MMU) [1]. Además, carece de acceso ambiental al host: toda interacción externa requiere importaciones explícitas [2]. Fuera del navegador, el WASI Subgroup del WebAssembly Community Group del W3C estandariza esa interfaz de sistema sobre el Component Model y WIT, con seguridad por capacidades [3] [4] [5]. WASI 0.3 añadió asincronismo nativo, y Wasmtime 46 fue la primera

versión en implementar su especificación final [3] [6]. El ecosistema incluye runtimes como Wasmtime, orquestadores como wasmCloud y frameworks como Spin [14] [15].

A diferencia del contenedor, que comparte el kernel pero empaqueta el espacio de usuario, Wasm aísla a nivel de bytecode dentro del mismo proceso, con una huella mínima y un arranque rápido [1] [11]. Mediciones revisadas por pares sitúan ese arranque bajo 10 ms con menos de 200 KB, y en cientos de microsegundos al restaurar instantáneas preinicializadas (sección 2.2) [12]. El costo es el rendimiento sostenido: sobre SPEC CPU, Wasm fue en promedio entre 1,45× (Firefox) y 1,55× (Chrome) más lento que el código nativo [13]. Como son mediciones de 2019 en navegadores y no en runtimes de servidor, deben leerse como orden de magnitud. En el borde, Fastly Compute ejecuta Wasm sobre Wasmtime [7] y Cloudflare Workers lo admite dentro de isolates de V8 [8], pero ambos exigen APIs propias: hostcalls en Fastly [9] y un soporte de WASI parcial y experimental en Cloudflare [10]. Así, la portabilidad del runtime no garantiza la de la aplicación (sección 2.6), y la mayoría de las toolchains aún apunta a WASI 0.2 [3].

Wasm conviene en cargas intermitentes u orientadas a eventos en el borde, donde su arranque y su huella aumentan la densidad de instancias por servidor y reducen costos mediante un escalado a cero efectivo [11] [12]. No conviene en cómputo intensivo de larga duración, donde los contenedores siempre encendidos mantienen superioridad en throughput [13], ni en proyectos que exigen estricta portabilidad entre plataformas, por las hostcalls propietarias, el soporte parcial de WASI y la madurez desigual de las toolchains [9] [10].

Referencias Bibliográficas

1. Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Gohman, D., Wagner, L., Zakai, A., Bastien, J. F., & Holman, M. (2017). Bringing the Web up to Speed with WebAssembly. <https://doi.org/10.1145/3062341.3062363>
2. W3C (2026). WebAssembly Core Specification (Candidate Recommendation Draft). World Wide Web Consortium. <https://www.w3.org/TR/wasm-core-2/>
3. WASI Subgroup (2026). WASI Releases. Fecha consulta: 20/09/2026 <https://wasi.dev/releases>
4. Bytecode Alliance; Gohman, D. / Bytecode Alliance (2024). WASI 0.2 Launched <https://bytecodealliance.org/articles/WASI-0.2>
5. WASI Subgroup (2024). WASI 0.2 Release Specification. Fecha consulta: 20/09/2026 <https://wasi.dev/releases/wasi-p2>
6. WASI Subgroup (11 de Junio de 2026). WASI 0.3 Release Specification. Fecha consulta: 20/09/2026 <https://wasi.dev/releases/wasi-p3>
7. Fastly (s.f.). Introduction to Compute. Fastly Documentation. Fecha consulta: 20/09/2026 <https://www.fastly.com/documentation/solutions/tutorials/compute/introduction-to-compute/1-introduction/>
8. Cloudflare (s.f.). How Workers works. Cloudflare Workers docs. Fecha consulta: 20/09/2026 <https://developers.cloudflare.com/workers/learning/how-workers-works>

9. Fastly (s.f.). Community-supported SDKs on the Compute platform. Fastly Documentation. Fecha consulta: 20/09/2026
<https://www.fastly.com/documentation/guides/compute/developer-guides/custom/>
10. Cloudflare (2026). WebAssembly (Wasm). Cloudflare Workers docs. Fecha consulta: 20/09/2026 <https://developers.cloudflare.com/workers/runtime-apis/webassembly/>
11. Kjorveziroski, V., Filiposka, S., & Trajkovik, V. (2021). IoT Serverless Computing at the Edge: A Systematic Mapping Review. Computers.
<https://doi.org/10.3390/computers10100130>
12. Shillaker, S., & Pietzuch, P. (2020). Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing. 2020 USENIX Annual Technical Conference (USENIX ATC 20), 419–433. <https://www.usenix.org/conference/atc20/presentation/shillaker>
13. Jangda, A., Powers, B., Berger, E. D., & Guha, A. (2019). Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code.
<https://www.usenix.org/conference/atc19/presentation/jangda>
14. CNCF (s.f.). wasmCloud. Cloud Native Computing Foundation. Fecha consulta: 20/09/2026. <https://www.cncf.io/projects/wasmcloud/>
15. Spin Framework (s.f.). Introducing Spin. CNCF. Fecha consulta: 20/09/2026.
<https://spinframework.dev/index>

2.6 Límites y dependencia del proveedor

3.1 Alternativas adicionales

Criterios inclusión y descarte

I1 -> vigencia y verificabilidad ,

- producto existe y se puede contratar.

- Q esté publicado: límites publicados, y precio publicado o declarado por el proveedor como solo por cotización

I2 -> pertenencia a la lista q tenemos q medir (funciones, contenedores sin server, computo borde, codigo abierto, datos sin server)

Regla operativa única: *“Ejecuta código (o consulta/almacena datos, en la ficha Datos) aportado por el usuario, bajo el modelo de cómputo/dato de esa lista, como servicio o runtime cuyo propósito principal es ese modelo — no un complemento de otro producto.”*

I3 -> Aporte diferencial con respecto a ficha original en 5 dimensiones:

unidad / modelo de cobro

aislamiento o escala a cero

proveedor no hiperescalar u otra jurisdicción

acoplamiento a otro servicio del mismo ecosistema

límites en otro orden de magnitud (timeout, payload, concurrencia)

X1 -> Redundancia de candidatos, dos candidatos proveen el mismo aporte con respecto a las plataformas base de la ficha.

Si dos candidatos aportan el mismo tipo de contraste frente a las plataformas base, se retiene el que cumpla, en este orden, el primer criterio que los separe: (1) precio de lista publicado frente a solo cotización o precio ausente; (2) límites técnicos publicados frente a no publicados; (3) disponibilidad general GA frente a beta / open beta / preview; (4) si sigue el empate, se retiene el de documentación oficial más específica para esa ficha (página de límites o pricing del propio servicio, no solo overview del producto) y se registra la elección en el anexo.

X2 -> Solapamiento o plataforma/servicio “extensión de” otro candidato

X3 -> Retirado, archivado o sin disponibilidad general (privado)

Funciones como servicio

1) Oracle cloud infrastructure functions

Fn project

IAM

pay as you go (invocation and execution pricing)

Integración con oracle cloud services

(muy usado en el mundo?)

- Timeout: 300 s en invocación síncrona, 3.600 s en detached (parámetros distintos)
- Memoria: valores fijos 128, 256, 512, 1024, 2048, 3072 MB
- Precio por tramos: invocaciones gratis hasta 2 millones/mes, después USD 0,0000002 c/u
- Tiempo de ejecución gratis hasta 400.000 GB-s/mes, después USD 0,00001417 por GB-s
- Concurrencia aprovisionada: se cobra al 25 % de la tarifa cuando NO se usa
- func.yaml y el CLI de Fn documentados como forma oficial de configurar el servicio

[Cambio de los valores por defecto de memoria y timeout](#)

2) Alibaba Function compute

Free trial quota of 150,000 CUs per cycle for three cycles. Each cycle lasts a month.

Subscription billing method y pay as you go

Development tools interesantes

integración con alibaba cloud services

menos usado/mas nicho/ china involved, desconfianza?

- Instancias en ráfaga: 300 en las regiones principales de China, 100 en el resto
- El exceso devuelve HTTP 429
- La versión vigente es FC 3.0; la API de FC 2.0

Fuente: [Link Alibaba](#)

3) IBM Cloud code engine functions

Functions dentro de “code engine”

Limits mas cortos? 120 s lenght of runtime, 100kb code inline

4)Scaleway Serverless Functions

Creación por UI, dashboard

API y CLI

proveedor europeo establecido, aunque mucho más chico que un hiperescalador.

- Timeout: 10 s a 60 min
- Payload: 6 MiB
- Tope de 50 instancias por función
- Escala a cero tras 15 min sin tráfico
- Proveedor europeo, precio en EUR

Fuente:[Functions limitations | Scaleway Documentation](#)

Puede ser candidato

Contenedores sin servidor

1) [fly.io](#)

fly machines viven en una fly app

CLI robusto para manejo de machines

scaling rápido d machines -> fly launch

integración apartado ia agents -> “sprites”

Cobro por segundo de máquina encendida

Empresa independiente, no hiperescalador

2) Railway

Startup chica: riesgo de continuidad

Cobra lo consumido, no lo reservado

- RAM USD 10/GB/mes y CPU USD 20/vCPU/mes
- Medidos por minuto, también mientras el servicio está en reposo
- Egress USD 0,05/GB

[Pricing Plans | Railway Docs](#)

3) Render

Las instancias del plan Free se apagan tras un periodo sin tráfico

Las instancias pagadas NO escalan a cero: el cómputo queda aprovisionado

Cobro: cuota de espacio de trabajo mas computo aprovisionado (planes Hobby, Pro, Scale)

En palabras simples: la misma plataforma se comporta distinto según el plan. En el gratuito el servicio se duerme; en el pagado queda encendido y se paga aunque no llegue trafico.

[Deploy for Free – Render Docs](#) [Render | The cloud for builders](#)

Cómputo en el borde

1) Akamai EdgeWorkers

Precio solo por cotización: la única así en el borde

Memoria muy baja: 1,5 a 4 MB por manejador

Orientado a grandes empresas (red de Akamai)

Tres niveles contratables de CPU (10/20/70 ms)

- Tres niveles: Basic, Dynamic, Enterprise Compute
- Memoria: 1,5 / 2,5 / 4 MB
- Tiempo total: 4 / 5,5 / 10 s
- PRECIO SOLO POR COTIZACIÓN (la tarifa se consulta con el representante)

[Resource tier limitations](#)

2) Supabase Edge Functions

Empresa en fuerte crecimiento (Serie F en 2026)

Integrada de fábrica con la base Postgres del mismo proyecto: URL de conexión incluida por defecto y permisos por usuario (Row Level Security)

Cruza borde + datos (relevante¿)

- Memoria 256 MB
- CPU 2 s por solicitud
- Vida del worker: 150 s en plan gratuito, 400 s en pago
- Corre sobre una plataforma de datos

[Limits | Supabase Docs](#)

[Environment variables | Supabase Docs](#)

[Integrating with Supabase Database \(Postgres\) | Supabase Docs](#)

3) Bunny Edge Scripting

Memoria: 128 MB

50 subsolicitudes

Script: máximo 10 MB; 128 variables de entorno

Arranque: 500 ms documentado

Precio: USD 0,02 por 1.000 s de CPU + USD 0,20 por millón de solicitudes

EL ANCHO DE BANDA CDN SE COBRA APARTE

30 segundos de procesador por solicitud, que es muchísimo para una plataforma de borde.

Y el compute se cobra separado del tráfico: la factura tiene dos líneas, no una.

[Limits - bunny.net Documentation](#) [Pricing - bunny.net Documentation](#)

4) Azure Front Door rules engine

Descarte

Motivo: no califica en la categoría. La documentación define cada regla como condiciones de coincidencia más hasta 5 acciones predefinidas (encabezados, redirecciones, reescritura de URL, caché, cambio de origen). No hay forma de ejecutar código propio.

[Edge Actions \(Preview\) - Azure Front Door | Microsoft Learn](#)

Código abierto

1) Fission

- Versión estable v1.27.0 del 22-06-2026; siete versiones entre mayo y junio de 2026
- El ejecutor por defecto (poolmgr) mantiene pods GENERICOS ya arrancados, 3 por defecto.

Al llegar la primera solicitud toma uno y le carga el código de la función

- poolmgr NO escala a cero; los otros dos ejecutores (newdeploy, container) si
- Se elige el ejecutor POR FUNCION

Fuente: [Ejecutor | Fission](#)

[Executors | Fission](#)

2) Nuclio

Versión 1.17.0 del 16-06-2026, con mantenimiento continuo

- Consume el stream el mismo: grupos de consumidores de Kafka, reparte particiones entre replicas, "a given partition is handled only by one replica", en orden
- Componente propio ("dealer") para repartir particiones o shards
- Disparadores: HTTP, Kafka, Kinesis, RabbitMQ, NATS, MQTT, Azure Event Hub, v3io, cron

Fuente: [Architecture](#)

[Kafka trigger](#)

3) KubeLess

Repositorio archivado por VMware el 15-12-2021. El README declara que ya no se mantiene.

[vmware-archive/kubeless: Kubernetes Native Serverless Framework](#)

DESCARTADO

4) Riff

Datos sin servidor

Repositorio archivado el 13-03-2021. El README declara el proyecto terminado.

[projectriff/riff: riff is for functions](#)

DESCARTADO

5) IronFunctions

No por abandono, sino por sucesión: el mismo equipo de Iron.io lo bifurcó en Fn Project al entrar a Oracle (2017). Fn es su continuación.

[The Fn Serverless Project is Now Available | by Travis Reeder | Fn Project | Medium](#)

IronFunction DESCARTADO

Datos sin servidor

1) Turso

- Corre sobre libSQL, bifurcación abierta de SQLite con licencia MIT:
"open source, open contribution fork of SQLite, created and maintained by Turso"
- libSQL incluye un servidor propio que se puede ejecutar FUERA de la nube de Turso
- La documentación dice que replica a cualquier ubicación, incluidos servidores propios
[tursodatabase/libsql: libSQL is a fork of SQLite that is both Open Source, and Open Contributions.](https://turso.tech/libsql/)

[libSQL - Turso](#)

2) Upstash

- USD 0,20 por 100.000 comandos
 - Máximo 10.000 comandos por segundo
 - En bases globales, cada escritura replicada cuenta como comando por región
- [Redis Pricing](#)

3) MongoDB Atlas Serverless

Sin creación de instancias nuevas desde febrero de 2025. Las existentes fueron migradas a clústeres Free, Flex o Dedicated, y el tipo dejó de tener soporte en 2025.

[Migrar herramientas programáticas a clústeres flexibles - Atlas - MongoDB Docs](#)

DESCARTADO

4) Firestore(google)

servicio de almacenamiento(db) escalable q vive en Google cloud

integración con firebase, cloud run functions

dos planes: estandar y enterprise

NoSQL

disponible en muchos lenguajes por SDKs, tiene API también

3.2 Cuadro comparativo: AWS y Azure · consolidación de la tabla

Criterios cerrados (costo, latencia/arranque en frío, complejidad operacional). Tabla AWS/Azure completa en [«3.2 – Tabla técnica y fuentes más abajo](#). Pendiente: consolidar con la mitad Google/Cloudflare/Fastly/Deno/open source (Claudio y Daniel).

3.2 Cuadro comparativo: Google, Cloudflare, Fastly, Deno/Vercel/Netlify, open source

4.1 Prueba de concepto

4.2 Modelo de costos

FRANCISCA ANTONIA ABARCA PEZO

4.3 Caso TERABYTE

5 Tendencias y riesgos

Borrador de apunte (2.2 Arranque en frío)

2.2 Arranque en frío

Causas del arranque en frío: Explicar el ciclo de vida de una instancia serverless cuando pasa de 0 a 1 (descarga del artefacto/imagen, aprovisionamiento del entorno/microVM/contenedor, inicio del runtime y ejecución del código de inicialización).

APUNTES EN BASE A “Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions”:

En la computación Serverless, específicamente Function as a Service (FaaS), a cada función se le asigna un contenedor para su ejecución. Estos contenedores se van liberando una vez ya han sido utilizados para evitar el desperdicio de recursos, esto se conoce como “escalado a cero” o “scale to zero”. Cuando los contenedores escalan a cero, estos se reinician al momento de que les llegan nuevas solicitudes del servidor. Cuando se están iniciando nuevamente para atender la función que deben ejecutar, provocan cierta latencia la cual es conocida como “arranque en frío” o “coldstart”.

Algunas causas de esto se debe a que hay que cargar nuevamente los entornos de ejecución y bibliotecas necesarias para ejecutar la función.

Otra causa es el entorno de ejecución como tal, por ejemplo HyperContainer presentaba la mayor latencia de arranque en frío, mientras que Docker basado en procesos presentaba la menor. También se comparó AWS Lambda y Microsoft Azure en donde la última ofrecía mejor rendimiento.

También, debido a su escalado dinámico, si se reciben muchas solicitudes al mismo tiempo, esto puede provocar retrasos en la escalabilidad misma.

Hay muchos factores como lenguaje de programación utilizado, dependencia de funciones, CPU y RAM.

Los lenguajes interpretados como Python, JavaScript y [Node.js](#) presentan menos latencia en arranque frío que lenguajes compilados como Java, C# y .NET. Pero los lenguajes de alto nivel como Python y Java presentan retrasos adicionales en el arranque debido a que tienen entornos de ejecución propios.

También influye la combinación entre dependencias y lenguajes de programación en específico, por ejemplo, AWS Lambda ejecuta Python más rápido que otras plataformas. En el paper se habla de más combinaciones comparando AWS Lambda, GCF, Azure con lenguajes como [Node.js](#), Python, C#, Java. El mejor en general que tiene tiempo de ejecución más rápido en la mayoría de los lenguajes es AWS.

También se analizaron las dependencias con distinto tamaño en GCF. En donde entre más dependencias, más latencia en el arranque frío había. El aprovisionamiento previo de las dependencias para las funciones de uso habitual puede reducir el número de arranques en frío.

Fases para pasar de 0 a 1:

-Fase de carga y preparación: Cuando se envía una solicitud de función, el contenedor debe prepararse para poder ejecutar dicha función. Se analizan los encabezados y los parámetros de entrada. Luego, se le asignan los recursos necesarios (CPU/RAM) para poder ejecutar la función. Para poder ejecutar la función como tal, se le asigna un contenedor, y este carga las dependencias necesarias para poder ejecutarla (bibliotecas, entorno de ejecución). Finalmente, se implementa el código de la función en el contenedor para ejecutarla.

-Ejecución de la función: Una vez completada la fase de carga y preparación del

contenedor, se ejecuta la función. Durante la ejecución, la plataforma supervisa y gestiona los recursos necesarios. Se interactúa con servicios externos como APIs. Finalizado el proceso de ejecución se envía la respuesta al cliente.

-Arranque en caliente: Luego de enviada la respuesta al cliente, se mantienen los recursos activos por cierta cantidad de tiempo para evitar el arranque en frío. Esto se conoce como “arranque en caliente”. Si no hay ninguna solicitud durante esa cantidad de tiempo, los contenedores se reinician a cero para ahorrar recursos.

Apuntes en base a “Benchmarking, Analysis, and Optimization of Serverless Function Snapshots”

En los modelos serverless,, la funcionalidad de la aplicación se divide en una o más tareas sin estado y controladas por eventos.

Para evitar el desperdicio de recursos, se limita la vida útil de las instancias entre 8 y 20 minutos, pasado ese tiempo, se tiene que crear una nueva instancia para llamar a la función.

Los principales proveedores de serverless, optaron por utilizar VMs como “sandbox” para garantizar la seguridad del entorno multitenant (multi - inquilinos, es decir, usan un mismo servidor para varios clientes).

Firecracker es un hipervisor diseñado por AWS el cual permite una capa de emulación mínima, en donde solo utiliza un unico tipo de dispositivo de bloque y un unico tipo de dispositivo de red, reduciendo el tiempo de arranque de las VMs a 125 ms y reduce el consumo de memoria del hipervisor a 3MB.

Otra opción para reducir los retrasos de arranques en frio son la creacion de instantaneas que capturan el estado actual de una maquina virtual, incluyendo el estado del monitor de la maquina virtual (VMM) y el contenido de la memoria fisica del invitado, almacenando como archivos en el disco. Cuando llega la solicitud de una funcion sin instancia pero con instantánea, el orquestador puede crear rapidamente una nueva instancia de esa funcion con la instantanea, reduciendo drasticamente la latencia de arranque en frio.

El problema de las instantaneas, o especificamente de Catalyzer, es que reducen el tiempo de inicializacion de las maquinas virtuales, pero trasladan el retraso en la ejecución del mismo código, ya que comienza a ejecutar el codigo antes de que la misma VMs esté completamente encendida, teniendo su RAM vacía y provocando errores de página.

Firecracker puso en practica lo mismo

La coincidencia entre Firecracker nativo y Catalyzer

Ambos sistemas dividen la restauración de la instantánea en dos fases:

1. Carga del VMM:Restauran el estado del hipervisor y los dispositivos emulados.
2. Paginación perezosa (Lazy Paging): Mapean el archivo de memoria pero dejan la RAM vacía, trayendo cada página desde el disco bajo demanda

“Por su parte, el procesamiento real de la función tarda mucho más (un 95 % más de media) en las invocaciones en frío en comparación con las invocaciones en caliente de las mismas funciones, llegando a alcanzar centenas de milisegundos incluso en el caso de funciones como «helloworld» y «pyaes», cuya ejecución en caliente solo requiere unos pocos milisegundos.”

También es importante que tan continuas están las páginas necesarias de cierto archivo, ya que si no tiene continuidad de páginas, se demora mucho más en encontrarla en la memoria y teniendo más probabilidad de fallos de página.

Mecanismos de mitigación:

- Analizar cómo los distintos proveedores atacan el problema: Instantáneas / Snapshots: AWS Lambda SnapStart.
- Concurrencia aprovisionada / Instancias mínimas: AWS Provisioned Concurrency, Google Cloud Run min instances, Azure Functions Premium plan.
- Aislamientos livianos: V8 isolates y Wasm (que enlaza con tu otra sección).

APUNTES EN BASE A “Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions”:

Soluciones basadas en caché:

Algunas mitigaciones utilizadas por los usuarios es acelerar el proceso de preparación de los contenedores, precargando las bibliotecas necesarias o de uso habitual.

También hay otra técnica que es cargar unicamente la parte esencial del código de la aplicación. Otra técnica es guardar la imágenes de las aplicaciones y ser capaz de compartirla con otros contenedores, de esta forma, se pueden restaurar y reducir en tiempo de latencia del arranque frío.

Los servicios sin servidor de Amazon, Microsoft y Google utilizan Amazon ElastiCache, MemoryStore y Azure Cache for Redis para acelerar el acceso a las aplicaciones.

Soluciones basadas en diseño:

WebAssembly es una opción de solución basada en diseño. También Catalyzer que es una nueva arquitectura con una rápida inicialización de funciones. Restaura la instancia de la función a partir de la imagen del punto de control reduciendo el coste de inicialización.

También hay otra opción que elimina los desequilibrios en las colas y reduce las interferencias, en donde se procesa solo el código de la función y sus dependencias sin necesidad de la imagen del SO, reduciendo latencia de arranque frío.

Soluciones basadas en el estado de ejecución de las funciones: Se almacena el estado de ejecución de una función localmente, ahorrando tiempo de latencia al momento de re-invocar la misma función basado en prebrake. La principal causa del retraso en el arranque en frío era la restauración de imagen de una función ya que llenaba la memoria del invitado. Cadden et al. propuso un método llamado SEUSS que permite la distribución rápida de funciones a partir de instantáneas del unikerne. También hay otro enfoque que es asignar funciones similares a los mismos contenedores mediante el algoritmo de sandbox. Las soluciones basadas en el estado de ejecución de las funciones varían en función de la arquitectura y el diseño de la plataforma sin servidor en la que se aplican.

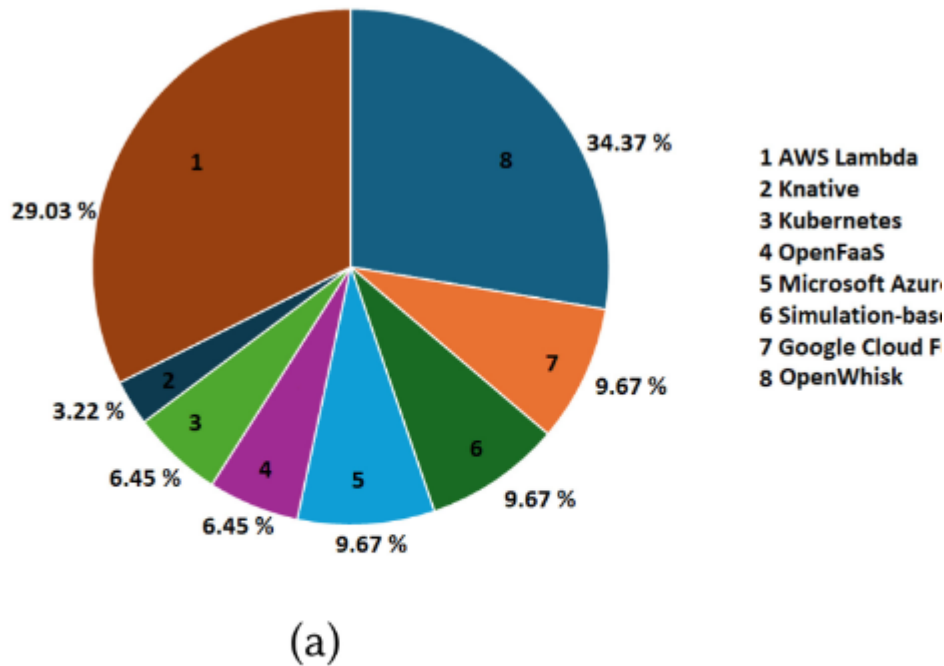


Fig. 15. (a) Percentage distribution of platforms. of code and dataset.

AWS Firecracker es una tecnología basada en microvmáquinas (microVM) y ofrece ventajas como una sobrecarga mínima y un funcionamiento más rápido para las instancias serverless en comparación con las máquinas virtuales tradicionales. Al tener sobrecarga mínima, el arranque en frío se reduce.

Tecnología gVisor propuesta por Google. gVisor actúa como una capa adicional entre la aplicación y el procesador, bloqueando las llamadas al sistema y reduciendo la latencia del arranque en frío, al tiempo que proporciona flexibilidad al ser compatible con los contenedores existentes.

Keeping Container Warm and Container Pool: Se basa en que los contenedores que ya fueron utilizados se mantienen funcionando durante un periodo de tiempo para evitar el arranque en cero. Esto puede funcionar pero desperdicia recursos.

Instance Prewarm-based studie: Se basa en tener la instancia de la función ya preparada antes de que llegue la solicitud, dificulta la gestión en aumentos repentinos de la demanda.

Opciones de ajuste dinámico para reducir arranque en frío según plataformas:

- AWS Lambda: Procesa los datos en el borde con la tecnología Lambda@Edge, debido a que los datos se procesan cerca de la fuente de datos se reduce el tiempo de respuesta en comparación a la comunicación entre usuario y servidor. Otra opción dinámica que ofrecen es optimizar las VPC que usan para la función lambda. Esto trae beneficios pero también costes adicionales y problemas de escalabilidad.
- Google Cloud Functions (GFC): Permiten reducir la tasa de escalado cuando se inician nuevas instancias. También permiten ajustar el número de instancias que pueden atender solicitudes. Esto también provoca desventajas como el coste y complejidad de la configuración.

- Microsoft Azure: Con el plan Premium permite mantener las funciones en estado “warm”, evitando el arranque en frío pero desperdiciando recursos.

Algunas opciones prometedoras para solucionar la latencia en arranque frío:

- Computación en el borde: Acerca los dispositivos informáticos a la fuente de datos, reduciendo latencia y uso innecesario de banda ancha.
- Redes definidas por software (SDN): Sistema de gestión de redes que configura dinámicamente la red, permite enrutar dinámicamente las llamadas a funciones en computación serverless.
- Tecnología de compiladores: Compilación anticipada de código para reducir latencia del arranque en frío.
- Predicción de recursos: Se predicen las llamadas a funciones.

Apuntes en base a “Benchmarking, Analysis, and Optimization of Serverless Function Snapshots”

Técnicas Fire-cracker, Cloud Hypervisor, virtualización a nivel de proceso, unikernels, cloudflare workers y faasm. También V8, WebAssembly y REAP.
Almacenamiento en caché con entornos de ejecución precalentados y preinicializados.

Dependencia clave con el equipo: Debes cerrar la sección 2.2 citando los resultados de la sección 4.1 (PoC de Vicente). La guía define que tu parte teórica debe rematar contrastando la teoría con el orden de magnitud medido empíricamente por el grupo (por ejemplo: “las mediciones propias del grupo en la sección 4.1 confirman este orden de magnitud...”)

Apuntes de SeBS:

AWS Lambda destruye **exactamente el 50% de los contenedores inactivos cada 380 segundos**. No importa si tu función está escrita en Python o Node.js, si tiene 128 MB o 1,5 GB de RAM, o si el paquete pesa 1 MB o 50 MB; la regla de los 380 segundos se aplica exactamente igual.

En los experimentos se pudo observar que en AWS Lambda su latencia de invocación tanto en caliente como en frío es bastante consistente y predecible. A diferencia de otras plataformas como Azure, GCP en donde su latencia de invocación en frío es bastante impredecible.

Apuntes de Google sobre CPU boost

Para evitar la latencia de arranque en frío, Google anunció para Cloud Run y Cloud Functions una nueva opción que permite asignar dinámicamente más CPU al contenedor al momento de arranque. Reduciendo hasta un 50% el tiempo de arranque en entornos de Java y hasta un 30% en entornos de [Node.js](#)

Blog AWS sobre facturación de fase de INIT

Se recomienda optimizar la carga de entorno de las funciones cargando los paquetes y bibliotecas realmente necesarios para la ejecución de código para no alargar el tiempo de descarga de este.

Presentan Lambda SnapStart la cual permite reducir la latencia de arranque frío sacando una instantánea de una función y reutilizándola para funciones posteriores, evitando ejecutar continuamente la fase INIT.

Concurrencia Aprovisionada: Se habla sobre esta técnica y se dice en qué tipos de entorno conviene usarla: “Esta estrategia de optimización es especialmente valiosa para funciones con requisitos de inicialización complejos o que gestionan cargas de trabajo sensibles a la latencia. Desde una perspectiva de optimización de costes, la Concurrencia Aprovisionada es más adecuada para cargas de trabajo con patrones de uso sostenidos superiores al 60 %, ya que esto suele proporcionar una mayor eficiencia de costes en comparación con la ejecución bajo demanda.”

Blog de AWS sobre SnapStart

(<https://docs.aws.amazon.com/lambda/latest/dg/snapstart.html>)

“SnapStart es particularmente beneficioso para dos tipos principales de aplicaciones:

API y flujos de usuario sensibles a la latencia: Las funciones que forman parte de puntos finales de API críticos o flujos orientados al usuario pueden beneficiarse de la latencia reducida y los tiempos de respuesta mejorados de SnapStart.

Flujos de trabajo de procesamiento de datos sensibles a la latencia: Los flujos de trabajo de procesamiento de datos con limitaciones de tiempo que utilizan funciones Lambda pueden lograr un mejor rendimiento al reducir la latencia de inicialización de funciones atípicas.”

“La concurrencia aprovisionada mantiene las funciones inicializadas y listas para responder en cuestión de milisegundos. Utilice la concurrencia aprovisionada si su aplicación tiene requisitos estrictos de latencia de arranque en frío que SnapStart no puede satisfacer adecuadamente.”

Blog de AWS sobre concurrencia aprovisionada

(<https://docs.aws.amazon.com/lambda/latest/dg/provisioned-concurrency.html>)

Se habla sobre:

Concurrencia reservada: Se reservan instancias mínimas y máximas exclusivas para una función en específico para asegurar de que siempre tenga disponible y asignada una.

Concurrencia aprovisionada: Se mantienen X cantidad de instancias mínimas activas para reducir la latencia de arranque en frío.

Flex Consumption Plan

(<https://learn.microsoft.com/en-us/azure/azure-functions/flex-consumption-plan>)

Ofrece un plan que incluye reducción de tiempo en arranque en frío con instancias siempre listas. Te cobran por Instancias en tiempo de ejecución + instancias siempre listas, te ofrecen hasta 1000 instancias de escalado horizontal. Solo compatible con Linux.

Plan Premium Azure Plan

(<https://learn.microsoft.com/en-us/azure/azure-functions/functions-premium-plan?tabs=portal>)

Ofrece instancias siempre listas y precalentadas, compatibilidad con tiempos de ejecución más prolongados. El plan Premium se factura según la cantidad de segundos de núcleo y memoria asignada a las instancias. No tiene cargos por ejecución. Tiene un costo mensual mínimo independientemente si tiene funciones activas o no. Las aplicaciones de funciones comparten las instancias asignadas y cada plan premium tiene como mínimo una instancia activa facturada.

Las instancias siempre listas son una categoría de instancias preasignadas que no se ven afectadas por el escalado, y las instancias precalentadas actúan como un búfer a medida que se escala debido a eventos HTTP.

VERSION BORRADOR:

2.2 Arranque en frío: causas, medición y mitigaciones

En los modelos serverless, se conoce como escalado a cero (scale to zero) cuando la plataforma libera los entornos de ejecución inactivos para no desperdiciar recursos; según mediciones de 2021, en AWS, cada 380 segundos se desaloja la mitad de los contenedores existentes, sin importar memoria, lenguaje ni tamaño del paquete [copik2021sebs]. Cuando llega una solicitud sin instancias disponibles, la plataforma debe reservar CPU y memoria, crear el entorno aislado (ver 2.1), cargar el runtime y las dependencias y ejecutar el código de inicialización [golec2025coldstart]. La latencia de esta transición se conoce como arranque en frío. Por otro lado, una invocación que encuentra una instancia ya preparada es un arranque en caliente. La demora aumenta con el tamaño de las dependencias y con runtimes de inicialización pesada como la JVM o .NET, frente a Python o Node.js [golec2025coldstart]. Además, no ocurre solo al salir de cero: también aparece al escalar de N a N+1 instancias durante ráfagas [giannini2022cpuboot].

Para medir el arranque en frío es preciso separar la fase de inicialización de la de ejecución, comparando invocaciones frías y calientes de una misma función mediante metodologías como Invoc-Overhead, la cual aísla la sobrecarga de la plataforma sincronizando marcas de tiempo entre cliente y servidor [copik2021sebs]. En AWS Lambda, cada invocación fría registra una duración de inicialización (Init Duration) diferenciada de la de ejecución; desde agosto de 2025 esta fase pasó a facturarse en todas las configuraciones, por lo que el arranque en frío, además de ser un problema de latencia, también es de costo [gupta2025initbilling].

Las mitigaciones actúan sobre distintas partes de este proceso y se diferencian en cuánto cuestan. La concurrencia aprovisionada evita que el arranque en frío ocurra, manteniendo instancias inicializadas antes de que llegue la demanda (Provisioned Concurrency en AWS, instancias mínimas en Cloud Run, instancias always ready en Azure Functions) [aws2023provisionedconcurrency, googlecloudrunmininstances, azurefunctionsflex]; sin embargo, se paga por capacidad ociosa, lo que acerca el modelo al de instancias siempre encendidas (ver 2.3 y 3.2), y no cubre el escalado de N a N+1, para lo cual Google ofrece el startup CPU boost [giannini2022cpuboot]. Las instantáneas, en cambio, se saltan la inicialización: AWS Lambda SnapStart guarda una captura cifrada de la microVM al publicar

la versión y restaura desde ella los nuevos entornos [aws2022snapstart]. Aun así, restaurar no es gratis, ya que la memoria se carga desde disco a medida que se usa y parte del retraso se traslada a la ejecución [ustiugov2021snapshots]. Por último, los entornos livianos como los aislamientos V8 y WebAssembly (ver 2.5) abaratan la creación del entorno: en un prototipo sobre OpenWhisk, Wasm redujo el arranque en frío en torno a un 94% frente a Docker [gackstatter2022pushing], aunque exige adaptar el código a la plataforma.

El impacto del arranque en frío depende del patrón de carga [copik2021sebs]. En APIs síncronas, degrada la experiencia de usuario. Para mitigar latencias intermitentes se usan instantáneas (como AWS Lambda SnapStart), asumiendo costos de almacenamiento en caché en Python y .NET [aws2022snapstart], o aislamientos livianos (V8/Wasm), lo que implica cambiar de plataforma [gackstatter2022pushing]. Aunque Cloudflare afirmó en 2020 haber eliminado el arranque en frío mediante handshake TLS [cloudflare2020workers], la evolución hacia scripts más pesados hizo que el problema reapareciera, aunque con su nueva técnica de Worker sharding lograron reducir los arranques en frío 10 veces [cloudflare2025shard].

En tráfico sostenido, usar concurrencia aprovisionada responde a costos: sobre el 60 % de uso continuo resulta más económica que la tarifa bajo demanda [gupta2025initbilling].

Finalmente, en procesamiento asíncrono en cola o tuberías de datos, la latencia inicial del arranque en frío es tolerable al no estar en la ruta crítica del usuario. La evidencia empírica de SeBS muestra que en funciones de larga duración (>10 s) la sobrecarga del cold start es porcentualmente despreciable frente al tiempo total de ejecución. Asimismo, en su análisis de punto de equilibrio, SeBS concluye que las alternativas siempre encendidas (máquinas virtuales IaaS) solo superan a serverless en rendimiento y precio si se garantiza una alta utilización continua; de lo contrario, el escalado a cero es más conveniente en costo para procesar cargas esporádicas [copik2021sebs].

Las mediciones propias del grupo (sección 4.1) [COMPLETAR: confirman / matizan] este orden de magnitud: [COMPLETAR: frío vs. caliente por plataforma, cuando Vicente tenga los datos].

Referencias:

1. [ustiugov2021snapshots]: @inproceedings{10.1145/3445814.3446714, author = {Ustiugov, Dmitrii and Petrov, Plamen and Kogias, Marios and Bugnion, Edouard and Grot, Boris}, title = {Benchmarking, analysis, and optimization of serverless function snapshots}, year = {2021}, isbn = {9781450383172}, publisher = {Association for Computing Machinery}, address = {New York, NY, USA}, url = {https://doi.org/10.1145/3445814.3446714}, doi = {10.1145/3445814.3446714}, booktitle = {Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems}, pages = {559–572}, numpages = {14},

keywords = {virtualization, snapshots, serverless, datacenters, cloud computing},
location = {Virtual, USA},
series = {ASPLOS '21}
}

2. [golec2025coldstart]: @article{10.1145/3700875,
author = {Golec, Muhammed and Walia, Guneet Kaur and Kumar, Mohit and Cuadrado, Felix and Gill, Sukhpal Singh and Uhlig, Steve},
title = {Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions},
year = {2024},
issue_date = {March 2025},
publisher = {Association for Computing Machinery},
address = {New York, NY, USA},
volume = {57},
number = {3},
issn = {0360-0300},
url = {https://doi.org/10.1145/3700875},
doi = {10.1145/3700875},
month = nov,
articleno = {65},
numpages = {36},
keywords = {Serverless computing, cold start latency, function as a service, survey, cloud computing}
}
3. [copik2021sebs]: @inproceedings{10.1145/3464298.3476133,
author = {Copik, Marcin and Kwasniewski, Grzegorz and Besta, Maciej and Podstawski, Michal and Hoefler, Torsten},
title = {SeBS: a serverless benchmark suite for function-as-a-service computing},
year = {2021},
isbn = {9781450385343},
publisher = {Association for Computing Machinery},
address = {New York, NY, USA},
url = {https://doi.org/10.1145/3464298.3476133},
doi = {10.1145/3464298.3476133},
booktitle = {Proceedings of the 22nd International Middleware Conference},
pages = {64–78},
numpages = {15},
keywords = {serverless, function-as-a-service, benchmark, FaaS},
location = {Quebec city, Canada},
series = {Middleware '21}}
4. [giannini2022cpuboot]: Giannini, S. (2022). New startup CPU boost improves cold starts in Cloud Run, Cloud Functions. Google Cloud Blog [giannini2022cpuboot].
5. [gupta2025initbilling]: Gupta, S., Gebhart, J. (2025). AWS Lambda standardizes billing for INIT Phase. AWS Compute Blog [gupta2025initbilling].
6. [aws2023provisionedconcurrency]: Amazon Web Services. (2023). Configuring provisioned concurrency for a function. AWS Lambda Developer Guide [aws2023provisionedconcurrency].

7. [googlecloudrunmininstances]: Google Cloud. (2026). Definir el número mínimo de instancias de los servicios. Documentación de Cloud Run [googlecloudrunmininstances].
8. [azurefunctionsflex]: Microsoft Learn. (2026). Azure Functions Flex Consumption plan hosting. Microsoft Azure Documentation [azurefunctionsflex].
9. [aws2022snapstart]: Amazon Web Services. (2022). Improving startup performance with Lambda SnapStart. AWS Lambda Developer Guide [aws2022snapstart].
10. [cloudflare2020workers]: Cloudflare Blog. (2020). Eliminating cold starts with Cloudflare Workers [cloudflare2020workers].
11. [cloudflare2025shard]: Hancock, H. (26 de septiembre de 2025). Eliminating Cold Starts 2: shard and conquer. Cloudflare Blog [cloudflare2025shard].
12. [gackstatter2022pushing]: @INPROCEEDINGS{9826054,

author={Gackstatter, Philipp and Frangoudis, Pantelis A. and Dustdar, Schahram},

booktitle={2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)},

title={Pushing Serverless to the Edge with WebAssembly Runtimes},

year={2022},

volume={},

number={},

pages={140-149},

keywords={Runtime environment;Computational modeling;Memory management;Serverless computing;Prototypes;Containers;Throughput;Function-as-a-Service;edge computing;server-less;WebAssembly},

doi={10.1109/CCGrid54584.2022.00023}}

3.2 – Tabla técnica y fuentes (borrador)

3.2 – Tabla técnica y fuentes (borrador)

Fuente de verdad: Borrador_3.2 v3 (20-09-2026), aceptado por Rodolfo. Filas = plataformas; columnas = campos. Región AWS us-east-1 · Azure East US · USD lista · consultado 20-09-2026. Aurora y DynamoDB salieron de 3.2 (van a 3.1/2.4/2.6). Criterios (costo, latencia/arranque en frío, complejidad operacional) cerrados por el equipo.

Tabla 3.2a — Criterios

Plataforma	Modelo	Aislamiento	Cobro y piso	Precio de lista	Arranque en frío	Mitigación frío	Qué opera el equipo	Límite que fuerza rediseño
AWS Lambda	FaaS	MicroVM (Firecracker) por invocación; entornos aislados por inquilino tarifados aparte (x86 0,000167 USD/entorno·GB; ARM 0,000133)	Por solicitud + GB-s; sin piso (escala a 0); cómputo escalonado por volumen	us-east-1: 0,20 USD/M solicitudes · x86 0,0000166667→0,000015→0,0000133334 USD/GB-s · Arm 0,0000133334→0,0000120001→0,0000106667 · free tier 1 M + 400.000 GB-s/mes + 100 GiB/mes streaming HTTP	Cualitativo <100 ms→1 s; sin p50/p95 acordado	SnapStart y Provisioned Concurrency (precios aparte; Arm más barato)	Runtime/código, IAM, despliegue, observabilidad, reintentos/DLQ; red si VPC (extrapolación desde shared responsibility AWS)	Timeout 900 s; payload síncrono 6 MB → partir tareas largas / mover a S3
AWS Fargate	Contenedores sin servidor	Por tarea, single-tenant, frontera de virtualización (AWS no nombra Firecracker en FAQs)	vCPU-s + GB-s desde pull hasta fin; mín. 1 min (Windows 5 min); 20 GB efímero incluidos; sin piso fijo	us-east-1: Linux/x86 0,0404784 USD/vCPU-h + 0,004446 USD/GB-h · Linux/ARM 0,0323798 USD/vCPU-h + 0,0035600 USD/GB-h	Medio-alto (pull + arranque); cualitativo	no documentado	Task def, VPC/ALB, auto scaling, imágenes, observabilidad; parcheo host lo hace AWS	Sin timeout de plataforma; estado no persistente → EFS/RDS/S3; combinaciones vCPU/memoria fijas
AWS App Runner 	Contenedores sin servidor	no documentado (no completar)	GB-h aprovisionadas (también en idle) +	us-east-1: activas 0,064 USD/vCPU-h + 0,007 USD/GB-h · aprovisionadas 0,007 USD/GB-h · cuota	Bajo con provisioned;	Instancias aprovisionadas	Muy bajo (build/deploy auto). Cerrado a nuevos	Timeout HTTP 120 s; 3 GB efímero;

	servidor	por analogía con Fargate)	vCPU-h/G B-h activas; mín. 1 min vCPU al activar; piso por defecto = 1 instancia aprovisionada	despliegues auto y build: monto no capturado	cualitativo	(es el piso)	clientes (vigente 20-09-2026); existentes siguen; AWS recomienda ECS Express Mode	estado no persistente
AWS Lambda@Edge	Borde	MicroVM como Lambda (extrapolación; AWS no lo declara aparte para @Edge)	Solicitud + GB-s (granularidad 1 ms); sin piso; sin free tier	Global (función se crea en us-east-1): 0,60 USD/M + 0,00005001 USD/GB-s	Mayor que CloudFront Functions; puede >1 s; cualitativo	no documentado y no disponible (sin Provisioned Concurrency ni SnapStart)	Como Lambda + restricciones: solo us-east-1; sin VPC, DLQ, capas, arm64, >512 MB /tmp; runtimes Node/Python recientes	Timeout 30 s; respuesta 40 KB viewer / 1 MB origin
AWS CloudFront Functions	Borde	Motor JS en edge; "isolate V8" = extrapolación (AWS solo lista restricciones del runtime)	Por invocación; free tier permanente; sin piso	Red global CloudFront: 0,10 USD/M invocaciones · Always Free 2 M/mes · (página también muestra ejemplo 0,01 USD/M — no usar; se usa 0,10)	Prácticamente nulo	No aplica	Solo función JS; muy limitado (sin red/FS/env/timers)	Función 10 KB; memoria 2 MB; respuesta 40 KB; sin estado
Azure Functions	FaaS	App Service VMs; aislamiento de cómputo declarado solo en Dedicated/ASE; Consumption/Flex/Premium: no documentado	Consumption/Flex On-Demand: ejecución + GB-s, sin piso · Flex Always Ready / Premium: piso (mín. 1 inst. en Premium)	East US: Cons. 0,20 USD/M + 0,000016 USD/GB-s · Flex OD 0,40 + 0,000026 · Flex AR 0,000004 baseline + 0,000016 + 0,40/M · Premium 126,29 USD/vCPU-mes + 8,979 USD/GB-mes	Cons.: cold real; Flex/Premium: always ready / warm; Dedicated ≥1: sin cold — cualitativo	Always ready / perpetually warm (se factura como piso)	PaaS Microsoft: SO Microsoft; app/red compartido; identidad/datos cliente (extrapolación a tareas del equipo)	HTTP duro 230 s (cualquier plan); request máx. 210 MB

Azure Container Apps	Contenedores sin servidor	Environment = secure boundary; Dedicated = pool reservado; tenancy Consumption: no documentado	Cons. activo: vCPU-s + GiB-s + /M req, sin piso si escala a 0; idle solo si réplicas mín.>0 y condiciones; Dedicated: piso gestión+vCPU+GiB	East US: Cons. activo 0,000024/vCPU-s + 0,000003/GiB-s + 0,40/M · idle 0,000003 · Dedicated 0,10 USD/h gestión + 0,0571/vCPU-h + 0,0050/GiB-h · free 180k vCPU-s + 360k GiB-s + 2 M req	Presente al escalar desde 0; magnitud pendiente	Réplicas mín.>0 (idle) o Dedicated → implica piso; sin SnapStart	Extrapolación CaaS/CNCF (Microsoft no nombra ACA en shared responsibility): perfiles, KEDA, imágenes, red, observabilidad	Sin timeout de plataforma; Cons. 0,25–4 vCPU / 0,5–8 GiB → Dedicated si memoria alta
Instancia siempre encendida (línea base)	Always-on	PENDIENTE (definir con Francisca: ¿EC2/Azure VM reservada u otra?)	PENDIENTE	PENDIENTE	No aplica	No aplica	PENDIENTE	PENDIENTE (sin límite tiempo/payload; contraste del argumento)

Tabla 3.2b — Límites técnicos y egreso

Plataforma	Tiempo máx. ejecución	Memoria / vCPU	Payload in/out	Concurrencia / escalado	Egreso
AWS Lambda	900 s (15 min); hasta 5.400 s async Managed Instances (salvo MQ/DocumentDB)	128 MB–10.240 MB; ~1.769 MB ≈ 1 vCPU; /tmp 512 MB–10.240 MB	Sínc. 6 MB req/resp; streaming resp. hasta 200 MB; async 1 MB	1.000 concurrentes cuenta/región (ajustable); burst 1.000 entornos/10 s	Remite a EC2 data transfer us-east-1: in 0; out Internet 100 GB/mes gratis luego 0,09→0,085→0,07→0,05 USD/GB; free same-region a S3/DynamoDB/etc.; a CloudFront 0
AWS Fargate	Sin timeout de plataforma	0,25–32 vCPU / hasta 244 GB (doc ECS); página Pricing solo lista hasta 16 vCPU/120 GB — se usa ECS	No aplica (contenedor)	ECS Service Auto Scaling; sin tope fijo (cuotas cuenta)	Misma tarifa EC2 [Ref 63]; AWS no publica egreso propio de Fargate

AWS App Runner ⚠	120 s HTTP; health máx. 5 min	0,25–4 vCPU / 0,5–12 GB	no documentado (más allá de ALB/WAF); 3 GB efímero	Autoscaling; 30 servicios/cuenta·región	Misma tarifa EC2 [Ref 63]; sin egreso propio App Runner
AWS Lambda @Edge	30 s (cuota general)	Igual que Lambda (128 MB–10.240 MB); sin arm64; /tmp ≤512 MB	Respuesta 40 KB viewer / 1 MB origin; include body truncado igual	1.000 concurrentes/región; 10.000 req/s/región; 10 req/s por instancia	CloudFront: Always Free 1 TB/mes out; luego US/MX/CA 0,085→0,080→0,060 USD/GB; origin fetch AWS gratis
AWS CloudFront Functions	no documentado como cuota de tiempo (solo “compute utilization” %)	Memoria función 2 MB	Función 10 KB máx.; respuesta 40 KB	100 funciones/cuenta; 100 distribuciones/función; tope concurrencia: no documentado	Igual CloudFront que Lambda@Edge
Azure Functions	Cons. 5–10 min; Flex/Premium/Dedicated: 30 min default, Unbounded; HTTP 230 s duro todos los planes	Cons. 1,5 GB fijo; Flex 512/2048/4096 MB; Premium 3,5/7/14 GB; Dedicated 1,75–256 GB (ASE)	Request máx. 210 MB; URL 8192; query 4096	Cons. 200 Win / 100 Lin; Flex 1.000 on-demand; Premium 100 Win / 20–100 Lin; Dedicated 10–30 (100 ASE)	Azure Bandwidth East US (no tarifa propia Functions): in gratis; out NAM/EU Premium 100 GB gratis luego 0,087→0,083→0,07→0,05; ISP routing más barato
Azure Container Apps	Sin timeout de plataforma	Cons. 0,25–4 vCPU / 0,5–8 GiB; Dedicated 4–32 vCPU / 16–128 GiB (GP) o 32–256 GiB (mem)	No aplica	Scale to zero opcional; rango exacto réplicas mín/máx: no documentado	Misma Azure Bandwidth [Ref 64]; sin egreso propio ACA
Instancia siempre encendida	—	PENDIENTE (Francisca)	—	—	PENDIENTE

Fuentes (Ref# planilla Referencias Investigación; consultadas 20-09-2026 salvo donde se indique):

- Ref 18 — AWS Lambda Quotas
- Ref 22 — Azure Functions Scale and Hosting (sin Clave BibTeX aún)

- Ref 25 — AWS Lambda Pricing
- Ref 32 — AWS Lambda Developer Guide
- Ref 48 — AWS Fargate Pricing
- Ref 49 — ECS CPU/memoria Fargate
- Ref 50 — AWS App Runner Pricing
- Ref 51 — App Runner Availability change
- Ref 52 — App Runner Architecture / develop
- Ref 53 — Azure Functions Pricing
- Ref 54 — Azure Container Apps Pricing
- Ref 55 — ACA Workload profiles
- Ref 56 — AWS Fargate FAQs
- Ref 57 — CloudFront Pay-as-you-go pricing
- Ref 58 — CloudFront Quotas
- Ref 59 — Lambda@Edge restrictions
- Ref 60 — CloudFront Functions restrictions
- Ref 61 — ACA environments
- Ref 62 — ACA billing
- Ref 63 — EC2 On-Demand (Transferencia de datos)
- Ref 64 — Azure Bandwidth Pricing
- Ref 65 — AWS shared responsibility (Lambda whitepaper)
- Ref 66 — Microsoft shared responsibility
- Ref 67 — CNCF Serverless Whitepaper 2018

Fuentes secundarias/cualitativas (no válidas para cifras duras, solo contexto de análisis)

Cloudflare Learning Center (comparativa de un proveedor sobre su propia categoría)

InventiveHQ "Serverless Showdown" (sin autoría técnica verificable) — Ref 33 descartada para egreso

SOBRE APP RUNNER: El 30 de abril AWS decidió cerrar el registro de nuevos usuarios para ese servicio (no se detalla en las fuentes las razones), para tenerlo en cuenta. Re-verificado vigente 20-09-2026 (Ref 51).

[Acceso al .md con la tabla completa.](#) →

https://drive.google.com/file/d/1nFV45DlnKF_fwLsdc7jBDEZJBtMjeD9V/view

Apuntes sección 2.5

Apuntes sección 2.5

WebAssembly y WASI como entorno de ejecución en el borde: portabilidad, aislamiento y arranque submilisegundo.

Apunte IoT Serverless Computing at the Edge:...

WebAssembly es una opción para cuando se buscan entornos de ejecución alternativos que no dependan de los containers, los que provocan la latencia de arranque en frío.

WebAssembly es una gran opción para dispositivos en el borde debido a su ligereza, portabilidad y rapidez del tiempo de inicio de sus funciones.

Si bien la nube es una excelente opción para aplicaciones que no requieran tiempos de respuesta tan rápidos, para los requerimientos de los procesamiento de datos de los dispositivos IoT es necesario optimizar esta lectura de datos.

Utilizar computación serverless en el borde logra combatir el paradigma de “enviar datos al código”, provocando una alta latencia de red, a ahora “enviar código a los datos”. Esto se logra preprocesando inicialmente los datos en el borde, ahorrando ancho de banda de la red y obteniendo tiempos de respuesta más rápidos, protegiendo a la vez los datos.

Con el objetivo de reducir la latencia del arranque en frío muchas plataformas reutilizan el mismo entorno, lo que deja archivos o procesos residuales.

Apuntes WASI 0.2 (<https://wasi.dev/releases/wasi-p2>)

Reemplazada por WASI 0.3

“Esto permite la modularidad, la virtualización (mediante la compatibilidad entre implementaciones) y la interoperabilidad entre lenguajes.”

WASI 0.2 tiene dos mundos: uno dirigido a programas de líneas de comandos otorgando acceso al sistema de archivos, variables de entornos y otras capacidades de la CLI. Por otro lado tiene uno dirigido a proxies HTTP con soporte para la transmisión simultánea de múltiples solicitudes HTTP..

Incluyen interfaces de E/S, Relojes, generación de números aleatorios seguros e inseguros, sistema de archivos, redes TCP y UDP, resolución DNS, CLI y manejo de solicitudes y respuestas HTTP para clientes y servidores.

Apuntes WASI 0.3 (<https://wasi.dev/releases/wasi-p3>)

Incorpora la asincronía nativa a los componentes WebAssembly.

Define 3 mundos. Uno para programas de línea de comandos con acceso al sistema de archivos, variables de entorno, entre otros. El segundo para servidores HTTP con clocks, random, cli stgio y la interfaz de cliente de HTTP. Reemplazando el mundo http/proxy de WASI 0.2. Finalmente tiene un mundo que extiende el anterior nombrado, importando la interfaz del controlador, para proporcionar soporte para los componentes de middleware de la ruta de solicitud.

Apuntes WASI 0.2 (<https://bytecodealliance.org/articles/WASI-0.2>)

Se basa en el modelo de componentes WASM, lo cual lo hace compatible con diferentes lenguajes sin una fragmentación drástica y virtualizable.

Incluye dos mundos: wasi-cli que es la interfaz de línea de comandos (archivos, sockets, relojes, números aleatorios, etc). wasi-http: mundo proxy HTTP, solicitudes y respuestas HTTP.

“Toda la E/S en Wasm pasa por sus importaciones y exportaciones, por lo que podemos virtualizar completamente la vista que un módulo Wasm tiene del exterior simplemente controlando a qué están vinculadas las importaciones y exportaciones.”

“ CloudABI combinaba la seguridad basada en capacidades con un trabajo impresionante para simplificar la plataforma Unix, reduciéndola a un conjunto relativamente pequeño de primitivas cuidadosamente diseñadas, lo que la hacía especialmente atractiva. En consecuencia, gran parte de WASI 0.1 se derivó directamente de CloudABI.”

Apuntes “Bringing the Web up to Speed with WebAssembly”

“WebAssembly es una abstracción sobre el hardware moderno, lo que lo hace independiente del lenguaje, del hardware y de la plataforma, con casos de uso que van más allá de la Web.”

WebAssembly aborda el problema de contar con código de bajo nivel seguro, rápido y portátil en la Web.

Qué debe tener un objetivo de compilación de bajo nivel?: Semántica segura, rápida y portátil de ejecutar independientemente del lenguaje, hardware y plataforma. Fácil de comprender, interoperabilidad sencilla con la plataforma web. Representación segura y eficiente, fácil de compilar y validar.

La semántica del acceso a la memoria es determinista y portátil en todos los motores y plataformas.

La memoria lineal está aislada por lo que programas compilados no pueden corromper el entorno de ejecución ni realizar comportamientos indefinidos. Esto permite interactuar de forma segura con JavaScript no confiable. Lograr un aislamiento rápido durante la ejecución de manera que se obtuviera un alto rendimiento.

El contenido de la pila de llamadas para la ejecución no se expone, por lo que un programa no puede WebAssembly en ejecución no puede acceder a él directamente.

“Está diseñado para que se pueda verificar de manera eficiente en una sola pasada lineal, intercalada con la decodificación binaria y la compilación.”

WebAssembly proporciona construcciones de flujo de control estructuradas parecidas a las de un lenguaje de programación. Esto permite que el código se valide de una sola pasada, se compile de una sola pasada o se transforme a una forma intermedia de tipo SSA en una sola pasada.

En arquitecturas de 64 bits, WebAssembly elimina las lentas comprobaciones de límites por software utilizando páginas de guarda en memoria virtual, lo que permite que el propio procesador (MMU) garantice el aislamiento a nivel de hardware sin perder velocidad de ejecución.

Borrador Version larga:

2.5 WebAssembly y WASI

WebAssembly (Wasm) se define como una abstracción portable sobre el hardware que ofrece una ejecución eficiente, segura e independiente de la plataforma [1] [2]. Su aislamiento en memoria se logra delimitando una memoria lineal separada del código y de la pila de ejecución [1]. En arquitecturas de 64 bits, Wasm elimina la sobrecarga de verificar límites por software asignando páginas de guarda en memoria virtual, lo que delega la protección al hardware (MMU) [1]. Asimismo, carece de acceso ambiental al sistema host, requiriendo importaciones explícitas para cualquier interacción externa [2].

Para estandarizar estas llamadas fuera del navegador, el WASI Subgroup del WebAssembly Community Group del W3C desarrolla WASI [3] sobre el Component Model y el lenguaje WIT, garantizando seguridad por capacidades [4] [5]. Por otro lado, WASI 0.3 incorporó asincronismo nativo, siendo Wasmtime 46 la primera versión en implementar su especificación final y ejecutar componentes 0.3 por defecto [3] [6]. El ecosistema incluye runtimes como Wasmtime, orquestadores como wasmCloud y frameworks como Fermyon Spin [14] [15]. En el borde, plataformas como Fastly Compute ejecutan Wasm de forma nativa sobre Wasmtime [7], mientras que Cloudflare Workers emplea isolates de V8 para admitir Wasm [8]. Aunque WASI estandariza la interfaz de sistema, las plataformas exponen APIs propietarias: Fastly exige sus propias hostcalls para ejecutar aplicaciones [9] y Cloudflare implementa WASI de forma experimental y parcial [10]. Por lo tanto, la portabilidad del runtime no garantiza la portabilidad de la aplicación, generando dependencia del proveedor (lock-in).

A diferencia de los contenedores tradicionales que comparten el kernel pero empaquetan el espacio de usuario y sus dependencias, Wasm aísla a nivel de bytecode dentro del mismo proceso, ofreciendo un consumo mínimo de memoria y un arranque rápido ideal para escalado a cero en routers o gateways de borde [1] [11]. Mediciones revisadas por pares sitúan ese arranque bajo 10 ms con una huella inferior a 200 KB, y solo alcanzan cientos de microsegundos al restaurar instantáneas preinicializadas [12], mitigación descrita en la sección 2.2. Sin embargo, Wasm no resulta conveniente para cargas CPU-intensivas de larga duración: un estudio empírico sobre SPEC CPU reporta una ralentización media de 1,45× en Firefox y 1,55× en Chrome (con picos de 2,08× y 2,5×) frente a código nativo, atribuida a optimizaciones faltantes, problemas de generación de código y rasgos inherentes a la plataforma [13]. Al tratarse de mediciones de 2019 en navegadores y no en runtimes de servidor como Wasmtime, deben leerse como orden de magnitud. En ese escenario, los contenedores siempre encendidos mantienen superioridad en throughput. Asimismo, la adopción del estándar vigente depende de la madurez de las toolchains: solo Rust cuenta con bindings para WASI 0.3, jco ofrece soporte experimental y la mayoría de las demás toolchains sigue apuntando a WASI 0.2 [3].

Wasm conviene cuando se procesan cargas intermitentes u orientadas a eventos en el borde, donde su tiempo de arranque inferior a 10 ms (o de cientos de microsegundos al restaurar instantáneas preinicializadas) y su huella de memoria menor a 200 KB aumentan la densidad de instancias por servidor y reducen costos mediante un escalado a cero efectivo [11] [12]. No conviene en tareas de cómputo intensivo de larga duración, donde pesa la ralentización media de 1,45×–1,55× frente a código nativo [13], ni en proyectos que exigen estricta portabilidad entre plataformas, debido a las hostcalls propietarias y al soporte parcial de WASI en algunos proveedores, sumados a la madurez aún desigual de las toolchains [3] [9] [10].